

Task 2

gRPC Lab

Overview

A .proto file was defined with four RPC methods declared: Unary, Server Streaming, Client Streaming and Bidirectional Streaming. This was then compiled to generate two python files, one for serializing and deserializing messages and the other with service stubs. A gRPC server was implemented which was containerized. A gRPC client was implemented to communicate with the containerized server. Two python files were created to send 10000 calls to REST container from Task 1 and to the gRPC server implemented to test their respective performances and to compare their respective results.

Github Repo: <https://github.com/bavanmendon/dsa-practicals/tree/master>

Preparation of .proto file

This file is the root of the gRPC implementation. This file is used to declare methods that will be used in the implementation. We have declared four RPC methods for Unary, Server Streaming, Client Streaming and Bidirectional Streaming of messages between two endpoints. It also contains format of the arguments of the RPC methods.

This file is then compiled using the following command in the directory where the implementation is located:

```
python3 -m grpc_tools.protoc \           // Runs protocol buffers compilation through python3
-I . \                                   // Imports from current working directory
--python_out=. \                         // Creates python classes in current directory for
                                         serialization and deserialization
--grpc_python_out=. \                   // Generates python classes for service stubs in the
                                         current directory

items.proto                             // The .proto files that needs to be compiled
```

This generates two files: filename_pb2.py and filename_pb2_grpc.py. The first file is used for serialization and deserialization of communicated data between two end points and the second file is used for calling python classes and service stubs.

gRPC Server Implementation

The two python files generated after compiling the protocol buffer are imported to this file. The four RPC methods are defined here to perform their respective functions on the server side. A variable called 'items' is created with test data.

- Unary: 1 request from items, 1 response to client
- Server Streaming: 1 request from items on server, response stream of items to client
- Client Streaming: transmit stream of items to add to server dataset, 1 response from server
- Bidirectional Streaming: transmit message stream to server, transmit the same message from server

A docker file is used to build and run server from the current working directory and to containerize it. This is connected to port 50051.

gRPC Client Implementation

The two python files generated after compiling protocol buffer are imported to this file as well. Stubs are referenced from the pb2-gRPC-python output file and the pb2-python file is used for serialization and deserialization of transmitted data across the two endpoints. Here the logic is written such that the stubs are used to call gRPC methods and also to send stream of data to the server. This program is run locally but connected to port 50051 to interact with the server.

Control Flow

Protocol buffer is complied to generate two python files for serialization/deserialization of streamed data and for implementing Python Classes and gRPC methods. Both these files are imported to the server and client apps. The gRPC file is basically a buffer that is linked to both the server and client. The server is open on port 50051 and containerized and the client is open locally and connected to port 50051. The client uses stubs to send requests to the server and the server responds with serializing this data. The client also serializes data when needed. This allows communication between the two endpoints.

Results

- This screenshot consists of compilation of protocol buffer to generate python classes, serialization / deserialization and stubs, building and running the docker file to execute the server and to containerize it.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-project/grpc-l
ab$ python3 -m grpc_tools.protoc -I . --python_out=. --grpc_python_out=. myitems.proto
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-project/grpc-l
ab$ docker build -t myitems:1.0 .
[+] Building 1.0s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 207B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.10.12  0.9s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/python:3.10.12@sha256:bac3a0e0d16125977e351c861e2f4b12ecafeaa6f72 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 11.44kB                                  0.0s
=> CACHED [2/5] WORKDIR /grpc-app                                  0.0s
=> CACHED [3/5] COPY requirements.txt                              0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> CACHED [5/5] COPY .                                             0.0s
=> exporting to image                                              0.0s
=> => exporting layers                                              0.0s
=> => writing image sha256:dc9a2425435c136e5ae95c03252dc64fe18c268ec588cdf670a3ef3579c79407 0.0s
=> => naming to docker.io/library/myitems:1.0                      0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-project/grpc-l
ab$ docker run -d -p 50051:50051 myitems-grpc:1.0
7ad7d86f85255b0e1ea5ac68afce2862ec6e2b5398760ccaf8fc640fe59ad87d
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-project/grpc-l
ab$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS
US            PORTS
7ad7d86f8525   myitems-grpc:1.0                   "python server.py"                     3 seconds ago Up 3
seconds      0.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp
5f0c26403507   ethersphere/bee:2.5.0              "bee start"                             5 weeks ago   Up 2
5 seconds    0.0.0.0:1633-1634->1633-1634/tcp
fddaad42e8ae   ghcr.io/sentinel-official/dvnpn-node:v0.7.1 "process start --log..."             5 weeks ago   Up 1
0 minutes    hivello/sentinel
```

- This screenshot shows the client being run locally to interact with the containerized server and to execute unary, server streaming, client streaming and bidirectional RPC methods.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-project/grpc-l
ab$ python3 client.py
/home/bavanmendon/.local/lib/python3.10/site-packages/google/protobuf/runtime_version.py:98: UserWarning:
Protobuf gencode version 5.29.0 is exactly one major version older than the runtime version 6.31.1 at my
items.proto. Please update the gencode to avoid compatibility violations in the next runtime release.
  warnings.warn(
=== Unary RPC: GetItemById ===
Unary response: id: 1
name: "Name 1"

=== Server-streaming RPC: ListAllItems ===
Server-streaming item: id: 1
name: "Name 1"

Server-streaming item: id: 2
name: "Name 2"

Server-streaming item: id: 3
name: "Name 3"

=== Client-streaming RPC: AddItems ===
Sending: Name 4
Sending: Name 5
Sending: Name 6
Client-streaming result: 3 items added
Client:Hello
Server: Received: Hello
Client:What's up?
Server: Received: What's up?
Client:Bidirectional testing
Server: Received: Bidirectional testing
```

Performance Tests

Task 1 is hosted on port 5000. Two python programs are written to send 10000 requests each to the REST service and gRPC service separately. The performance efficiency is checked using this test.

REST Performance

GET request is sent 10000 times on port 5000 and the result is as given below:

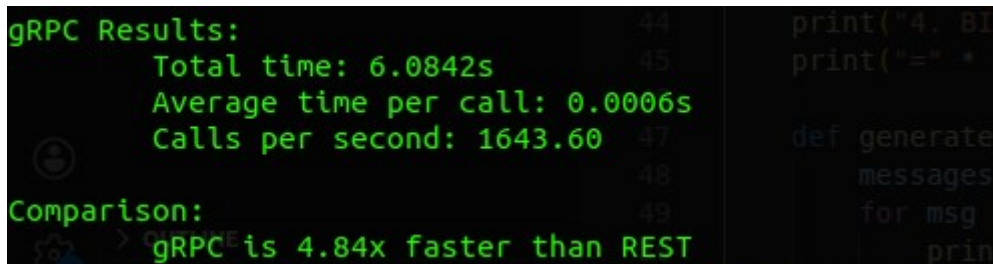
```
Performance Comparison: REST vs gRPC
=====
Time per request: 0.0028781890869140625
REST API Results:
    Total time: 29.4551s
    Average time per call: 0.0029s
    Calls per seconds: 339.50s
```

Total time: 29.4551 seconds

Average time per request: 0.002 seconds

gRPC Performance

stub.GetItemId() request is sent 10000 times on port 50051 and the result is as given below:



The screenshot shows a terminal window with gRPC performance results on the left and a code snippet on the right. The results indicate a total time of 6.0842s, an average time per call of 0.0006s, and 1643.60 calls per second. A comparison states that gRPC is 4.84x faster than REST. The code snippet on the right shows a Python function named 'generate' that iterates over messages and prints them.

```
gRPC Results:
    Total time: 6.0842s
    Average time per call: 0.0006s
    Calls per second: 1643.60
Comparison:
    gRPC is 4.84x faster than REST

print("4. BI
print("=" *
```

Total Time: 6.0842 seconds

Average time per request: 0.0006 seconds

Comparison

As we can see, the gRPC service performs 4 – 5 times better than the REST service. The REST service interacts directly the host. Whereas, the gRPC service acts as a middle ground for communication between client and server.

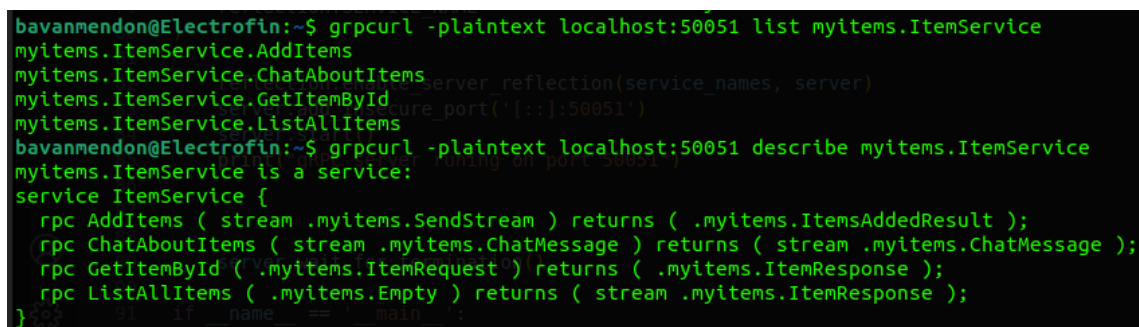
Reflection & Interceptors

The logging interceptor is defined to intercept all incoming requests to the server before reaching the service handlers. After adding the reflection and interceptors, the container with the server is rebuilt and rerun.

An example to list all methods can be shown using this command:

```
grpcurl -plaintext localhost:50051 list myitems.ItemService
```

The screenshot below shows the output from two types of commands that show methods in 'myitems' and describes the structure of the methods as well using the same command with different parameters.

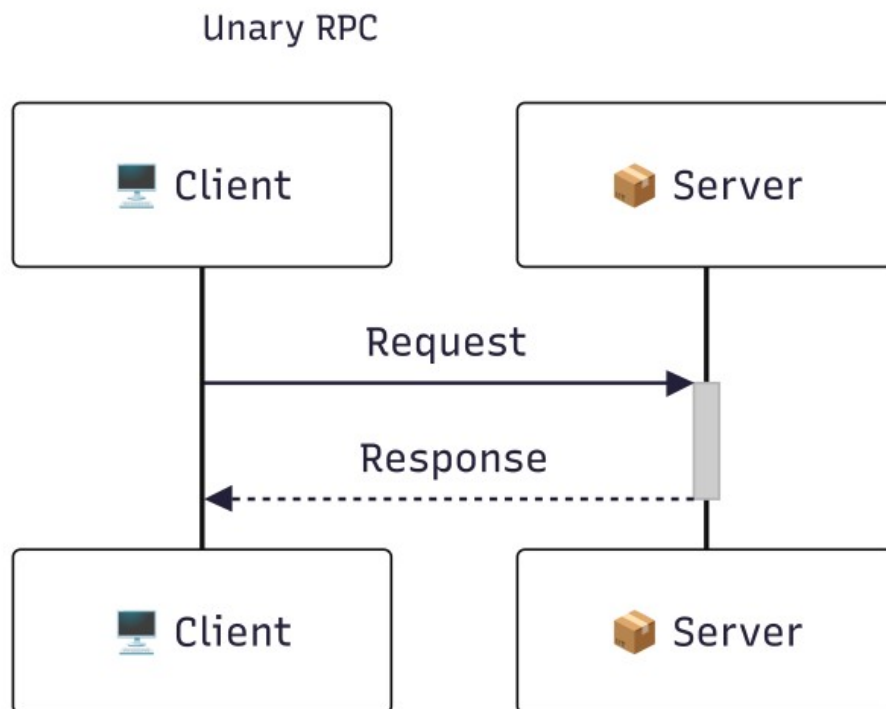


The screenshot shows a terminal window with two grpcurl commands and their output. The first command lists the methods of the myitems.ItemService, and the second command describes the structure of the methods. The output shows the service name and the methods it supports, including AddItems, ChatAboutItems, GetItemById, and ListAllItems.

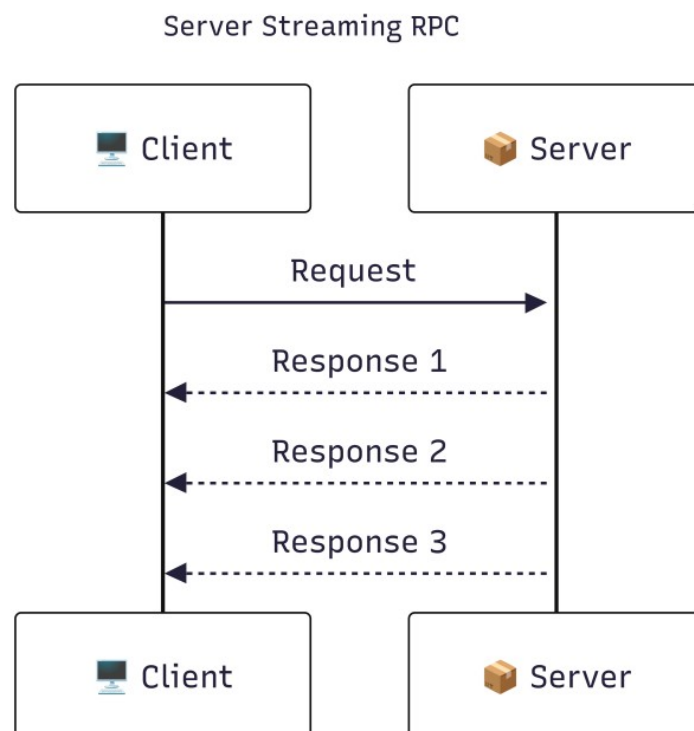
```
bavanmendon@Electrofin:~$ grpcurl -plaintext localhost:50051 list myitems.ItemService
myitems.ItemService.AddItems
myitems.ItemService.ChatAboutItems
myitems.ItemService.GetItemById
myitems.ItemService.ListAllItems
bavanmendon@Electrofin:~$ grpcurl -plaintext localhost:50051 describe myitems.ItemService
myitems.ItemService is a service:
service ItemService {
  rpc AddItems ( stream .myitems.SendStream ) returns ( .myitems.ItemsAddedResult );
  rpc ChatAboutItems ( stream .myitems.ChatMessage ) returns ( stream .myitems.ChatMessage );
  rpc GetItemById ( .myitems.ItemRequest ) returns ( .myitems.ItemResponse );
  rpc ListAllItems ( .myitems.Empty ) returns ( stream .myitems.ItemResponse );
}
```

Sequence Diagrams

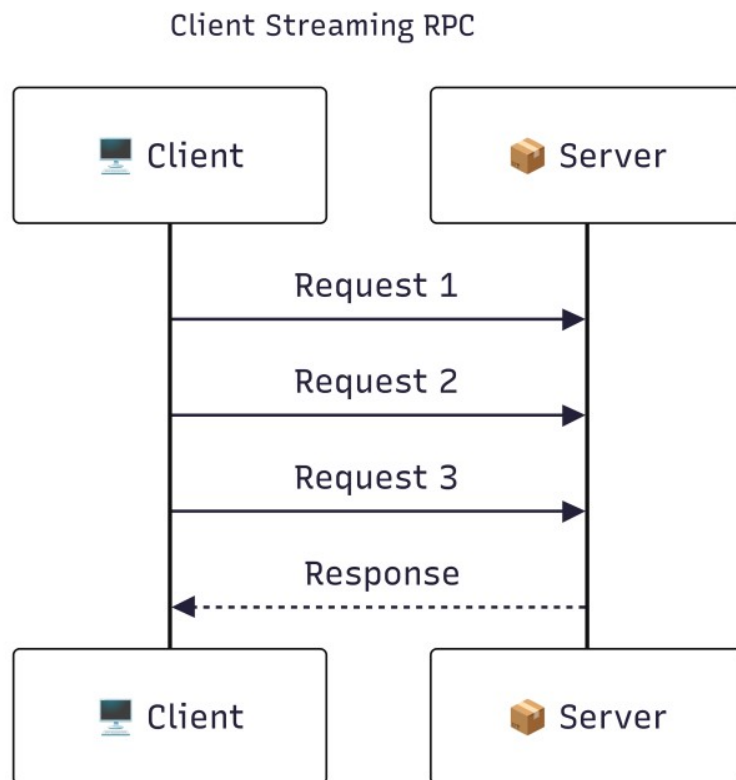
Unary



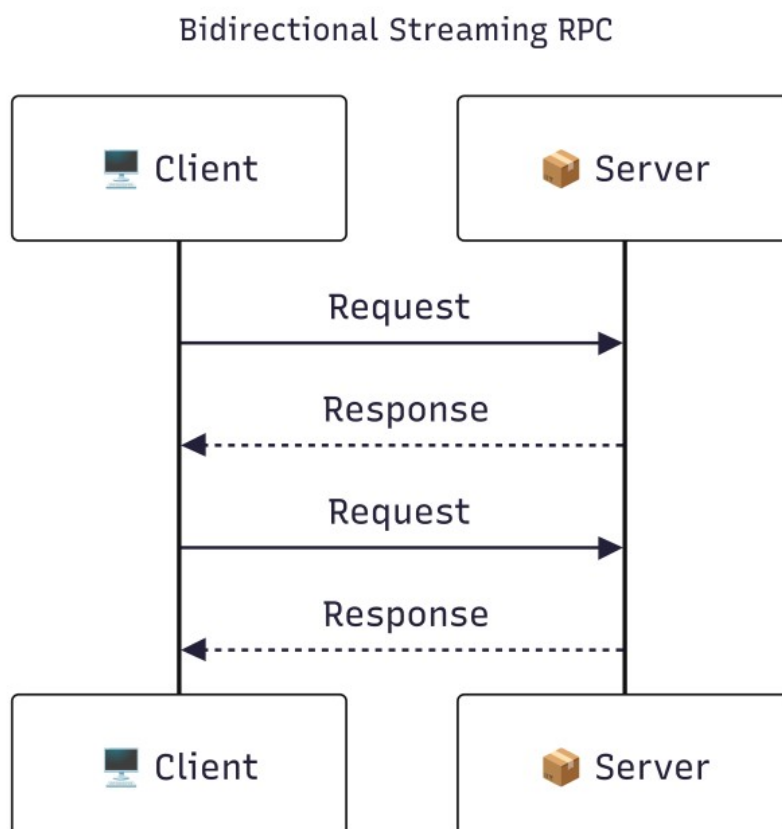
Server Streaming



Client Streaming



Bidirectional Streaming



Approach to Error Handling, Reflection, and Interceptors

Error handling in gRPC was managed by returning well-defined status codes and descriptive error messages, making it easier for clients to handle failures programmatically. Reflection was enabled using the `grpcio-reflection` package, which allowed for dynamic inspection and troubleshooting with tools like `grpcurl`. Logging interceptors were implemented by subclassing `grpc.ServerInterceptor`, providing visibility into incoming requests and metadata. These tools and practices improved both development efficiency and runtime observability.

Potential Real-World Uses of Streaming

Streaming in gRPC is well-suited for scenarios where large datasets or continuous data flows are required. Examples include real-time chat applications, live telemetry feeds, video streaming, or any system where updates need to be pushed to clients as they happen. Server-side streaming can efficiently deliver results as they become available, while bidirectional streaming enables interactive, low-latency communication.